



Medical Imaging Systems- Assignment 1

AUTHORS

Heshan Harshana Jayasinghe Arachchige - s242511
Leonel Vazquez Carrasco - s251976

October 26, 2025

Contents

1	Abstract	1
2	Introduction	1
3	Methodology	1
3.1	Beamforming Algorithm Overview	2
3.2	Convex Array Transducer Geometry	3
3.3	Beamforming Principle: Delay-and-Sum	4
3.3.1	Fixed Received Focusing	6
3.3.2	Dynamic Received Focusing	7
3.4	Signal Interpolation for Fractional Delays	8
3.5	Apodization	9
3.6	Envelope Detection and Log Compression	10
3.7	Scan Conversion: Polar to Cartesian Interpolation	10
3.8	Quantitative Analysis of Lateral Resolution	11
4	Results	13
4.1	Beamforming of Simulated Point Phantom	13
4.1.1	Impact of Receive Focusing Method	13
4.1.2	Impact of Apodization Window	14
4.1.3	Quantitative Lateral Resolution Analysis	15
4.2	Beamforming of Measured Phantom Data	17
4.3	Beamforming of <i>In-Vivo</i> Liver Data	17
5	Discussion	19
6	Conclusion	20
7	Appendix	21

1 Abstract

This report details the implementation of a Delay-and-Sum (DAS) beamformer in MATLAB for a convex array ultrasound transducer. The beamformer's performance was systematically evaluated by analyzing the critical impact of receive focusing and apodization on image quality. Results from a simulated wire phantom demonstrate that dynamic receive focusing is essential to maintain uniform lateral resolution across depth, unlike fixed focusing which suffers from severe defocusing artifacts. Furthermore, apodization windows present a fundamental trade-off: while rectangular windows offer the best theoretical resolution, moderate windows like Hann provide superior image contrast by suppressing side lobes, offering an optimal balance. The beamformer was successfully validated on measured phantom and in-vivo liver data, producing clinically relevant images and confirming the practical implications of these beamforming parameters.

2 Introduction

Ultrasound beamforming is the foundational signal processing technique that forms diagnostic images from raw channel data. The DAS algorithm is its core, combining time-delayed signals from multiple transducer elements to coherently synthesize ultrasound beams. While the principles are well-established for linear arrays, the curved geometry of a convex array introduces distinct challenges for calculating these time delays and forming an image.

This work implements a DAS beamformer in MATLAB specifically designed for a convex array transducer. The primary objective is to experimentally investigate and quantify the impact of two critical beamforming parameters on image quality: the method of receive focusing (dynamic versus fixed) and the choice of apodization window. The analysis is grounded in quantitative metrics, specifically lateral resolution measured via the Full Width at Half Maximum (FWHM) of point targets.

The report is structured as follows: Section 3 details the beamformer implementation and theoretical foundations. Section 4 presents a comparative analysis of the beamforming parameters using simulated, measured, and in-vivo data. Section 5 discusses the results and their practical implications, followed by conclusions in Section 6.

3 Methodology

This section delineates the end-to-end implementation of the delay-and-sum beamformer for convex array ultrasound data. The process is systematically broken down, beginning with the definition of the transducer's geometry and culminating in the generation of a rectilinear B-mode image. Each subsequent subsection provides a detailed exposition of a critical component, including the core beamforming principle, dynamic and fixed focusing techniques, apodization, and essential post-processing steps. The theoretical foundations are consistently paired with their practical implementation in the accompanying MATLAB code.

3.1 Beamforming Algorithm Overview

The core of this project is the implementation of a Delay-and-Sum beamformer for a convex array transducer. The high-level workflow, implemented in the MATLAB function `beamform`, is illustrated in Figure 1 and can be summarized as a multi-stage processing chain.

The process begins by defining the unique geometry of the convex array, which is critical for all subsequent delay calculations (Section 3.2). For each of the 129 emissions, which correspond to individual scan lines, the following steps are performed: the RF data from the 64-element active aperture is loaded; for every depth sample along that line, dynamic receive focusing is applied by calculating unique time delays for each element (Section 3.3); these delays are applied using linear interpolation (Section 3.4); and the signals are weighted by an apodization function (Section 3.5) before summation.

Finally, the beamformed RF data is converted into a displayable B-mode image through envelope detection via the Hilbert transform, log compression to a 60 dB dynamic range, and scan conversion from polar to Cartesian coordinates (Section 3.7).

This modular workflow allows for the isolated investigation of key parameters, such as focusing and apodization, the details of which are elaborated in the following subsections.

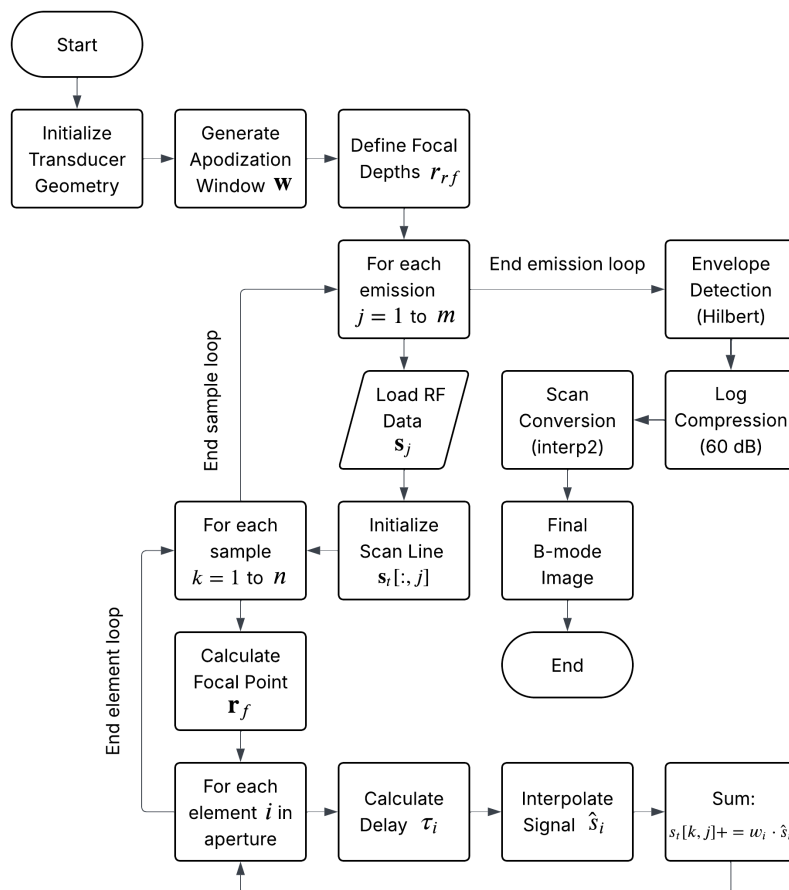


Figure 1: High-level flowchart of the implemented convex array beamforming algorithm.

3.2 Convex Array Transducer Geometry

The beamforming algorithm is fundamentally governed by the physical geometry of the convex array transducer. Accurate calculation of the spatial position of each element is paramount, as these coordinates are the reference points for all subsequent time-delay calculations. The complete set of transducer parameters used in this work is provided in Table 1.

Table 1: Convex Array Transducer Parameters

Parameter	Value	Unit
Number of Elements	192	–
Transducer Center Frequency	3.50	MHz
Number of Active Elements in Transmit	64	–
Element Height	13	mm
Element Width	0.30	mm
Kerf	0.03	mm
Convex Radius	60.3	mm
Elevation Focus	65	mm

The first step in defining the geometry is to calculate the center-to-center distance between adjacent elements, known as the pitch (p). This is the sum of the element width and the kerf:

$$p = w_{\text{element}} + w_{\text{kerf}} = 0.30 \text{ mm} + 0.03 \text{ mm} = 0.33 \text{ mm}. \quad (1)$$

The total arc length (L_{total}) spanned by the $N = 192$ elements is then simply the number of elements multiplied by the pitch:

$$L_{\text{total}} = N \cdot p. \quad (2)$$

Assuming the elements are evenly distributed along the circular arc, the total angular span (θ_{total}) subtended by the array can be found by the relation between the arc length and the radius (R):

$$\theta_{\text{total}} = \frac{L_{\text{total}}}{R}. \quad (3)$$

To calculate the angular position of an individual element i , we define a coordinate system where the origin 0 is at the center of curvature of the array, and the z -axis is the symmetry axis of the transducer, pointing into the medium. The angular position θ_i of the i -th element, where $i \in [1, N]$, is given by:

$$\theta_i = \left(i - \frac{1}{2}\right) \frac{\theta_{\text{total}}}{N} - \frac{\theta_{\text{total}}}{2}. \quad (4)$$

This equation places the first element at an angle of $-\theta_{\text{total}}/2$ (left-most edge), the last element at $+\theta_{\text{total}}/2$ (right-most edge), and each element at the center of its respective pitch.

Finally, the Cartesian coordinates $\mathbf{r}_i = (x_i, z_i)$ for each element can be derived from the simple polar-to-Cartesian transformation:

$$\begin{aligned} x_i &= R \cdot \sin(\theta_i), \\ z_i &= R \cdot \cos(\theta_i). \end{aligned} \tag{5}$$

This formulation places the transducer elements on a circle of radius R , with the convex face pointing in the positive z -direction. Figure 2 provides a visual representation of this geometry, which is essential for intuitively understanding the beamforming process for a curved array.

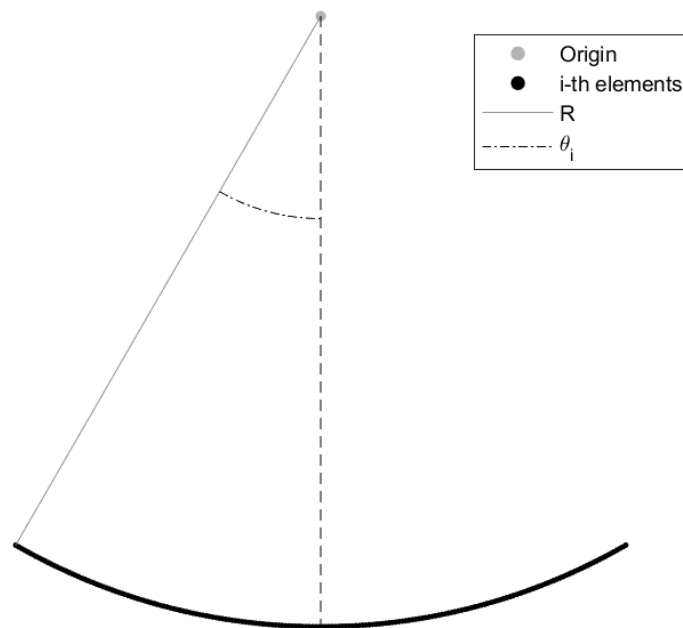


Figure 2: Schematic of the convex array transducer geometry, showing the element distribution along an arc of radius R and the calculated angular positions θ_i .

3.3 Beamforming Principle: Delay-and-Sum

The fundamental objective of beamforming is to combine the radio-frequency (RF) signals received by the individual transducer elements in such a way that echoes originating from a specific focal point $\mathbf{r}_f$ are reinforced, while signals from other directions are attenuated. The Delay-and-Sum (DAS) algorithm achieves this by applying time-dependent delays to the received signals before summing them, effectively synchronizing the wavefronts from the desired point.

The geometry for this process is illustrated in Figure 3. For a given focal point $\mathbf{r}_f$, the time it takes for an acoustic wave to travel from $\mathbf{r}_f$ to the i -th element located at $\mathbf{r}_i$ is

proportional to the distance between them. The propagation delay τ_i for the i -th element is calculated as:

$$\tau_i(\mathbf{r}_f) = \frac{|\mathbf{r}_f - \mathbf{r}_i| - |\mathbf{r}_f - \mathbf{r}_c|}{c}. \quad (6)$$

The terms in this equation are defined as follows:

- $\mathbf{r}_f = (x_f, z_f)$: The spatial coordinates of the focal point currently being beamformed.
- $\mathbf{r}_i = (x_i, z_i)$: The spatial coordinates of the i -th receiving element, derived from Equation (5).
- $\mathbf{r}_c = (x_c, z_c)$: The spatial coordinates of the the center of the active aperture (beam reference).
- $|\mathbf{r}_f - \mathbf{r}_i|$: The Euclidean distance from the focal point to the i -th element. This represents the total propagation path of the echo.
- $|\mathbf{r}_f - \mathbf{r}_c|$: The Euclidean distance from the focal point to the beam reference. Subtracting this ensures that the delays are relative, aligning the signals instead of absolutely shifting them.
- c : The speed of sound in the medium (1491 m/s in this work).

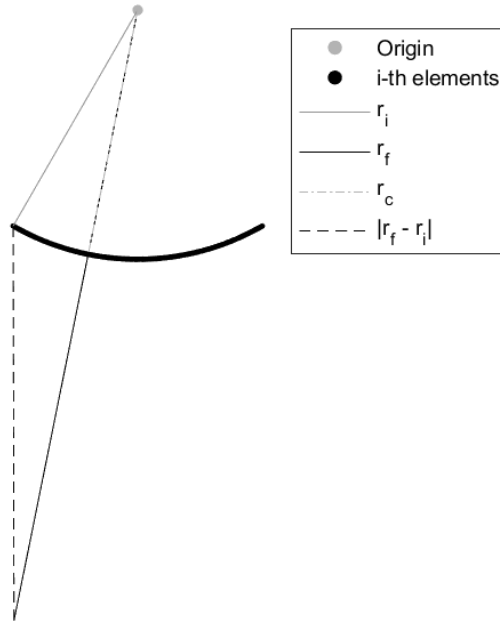


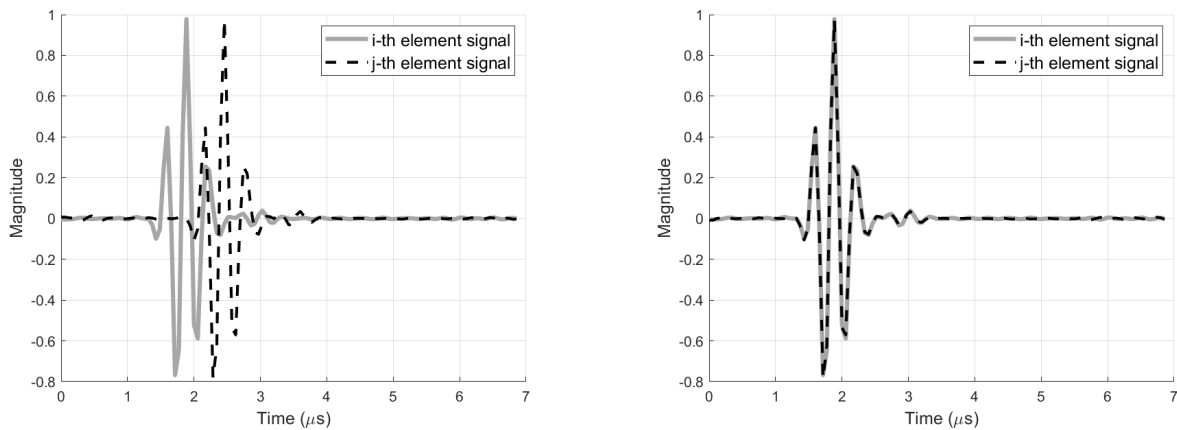
Figure 3: Geometry for delay calculation in a convex array. The delay for the i -th element is based on the difference between the propagation path $|\mathbf{r}_f - \mathbf{r}_i|$ and a reference path $|\mathbf{r}_f - \mathbf{r}_c|$.

Once the time delays τ_i are calculated for all N_{active} elements in the receive aperture, the delayed signals are summed to form the beamformed signal s_{BF} for the focal point $\mathbf{r}_f$:

$$s_{\text{BF}}(t, \mathbf{r}_f) = \sum_{i=1}^{N_{\text{active}}} w_i \cdot s_i(t - \tau_i(\mathbf{r}_f)). \quad (7)$$

Here, $s_i(t)$ is the RF signal received by the i -th element, and w_i is the apodization weight assigned to that element, which will be discussed in Section 3.5.

The effect of this process is conceptually illustrated in Figure 4. A wavefront emanating from a point scatterer will arrive at each element at a different time. Without applying delays (Figure 4a), the signals are desynchronized, and their sum results in a low-amplitude, distorted signal due to destructive interference. After applying the correct geometrical delays (Figure 4b), the signals become synchronized, and their summation results in a high-amplitude, coherent signal, thereby significantly improving the sensitivity and resolution at the focal point.



(a) Desynchronized signals before delay correction.

(b) Synchronized signals after applying correct delays.

Figure 4: Conceptual illustration of the Delay-and-Sum principle. (a) Without delays, signals from a point target sum destructively. (b) With proper delays, they align and sum constructively, enhancing the signal.

This general DAS principle is applied along each scan line to reconstruct the entire image. The specific methods for determining the focal point $\mathbf{r}_f$ and the reference distance $|\mathbf{r}_f - \mathbf{r}_c|$ distinguish the fixed and dynamic focusing approaches, which are detailed in the following subsections.

3.3.1 Fixed Received Focusing

In fixed receive focusing, a single, constant focal depth is used for an entire scan line. This approach is computationally simpler but results in suboptimal focusing away from the

chosen focal depth. The focal point $\mathbf{r}_f$ for all samples along a scan line is defined at a fixed radial distance r_{fixed} from the origin, along the direction of the scan line θ_{rf} (the mean angle of the active aperture).

For a sample index k corresponding to a radial distance r_k , the focal point is calculated as:

$$\begin{aligned} x_f &= r_{\text{fixed}} \cdot \sin(\theta_{\text{rf}}), \\ z_f &= r_{\text{fixed}} \cdot \cos(\theta_{\text{rf}}). \end{aligned} \quad (8)$$

In this method, the reference distance $|\mathbf{r}_f - \mathbf{r}_c|$ in the general delay equation (Eq. (6)) is set to the distance from the focal point to the center of the active aperture $\mathbf{r}_c$. This ensures that the signal from the focal point itself will be perfectly in phase. However, for points closer to or farther from the transducer than r_{fixed} , the wavefront curvature does not match the applied delays, leading to defocusing and a broadening of the point spread function (PSF). This manifests in the image as reduced lateral resolution in the near and far fields.

The algorithmic implementation of fixed focusing is straightforward. The vector of focal radii $\mathbf{r}_{rf}$ for all n samples is simply a constant array:

$$\mathbf{r}_{rf} = [r_{\text{fixed}}, r_{\text{fixed}}, \dots, r_{\text{fixed}}]. \quad (9)$$

In the provided code, r_{fixed} was chosen to be 40 mm beyond the transducer radius, i.e., $r_{\text{fixed}} = R + 40$ mm.

3.3.2 Dynamic Received Focusing

Dynamic receive focusing (DRF) overcomes the limitation of fixed focusing by updating the focal point for every single sample along the scan line. This ensures that the beamformer is perfectly focused at every depth, yielding superior image resolution throughout the entire field of view.

In DRF, the focal point $\mathbf{r}_f$ for the k -th sample is defined to lie at the precise radial distance corresponding to that sample's time-of-flight. The radial distance for the k -th sample is given by:

$$r_k = R + \frac{k}{f_s} \cdot \frac{c}{2}, \quad (10)$$

where f_s is the sampling frequency, c is the speed of sound, and the factor of 2 accounts for the round-trip travel of the echo. The focal point coordinates are then:

$$\begin{aligned} x_f &= r_k \cdot \sin(\theta_{\text{rf}}), \\ z_f &= r_k \cdot \cos(\theta_{\text{rf}}). \end{aligned} \quad (11)$$

Crucially, in dynamic focusing, the reference distance $|\mathbf{r}_f - \mathbf{r}_c|$ in Eq. (6) is set to zero. This is because the focal depth is now exactly equal to the depth of the sample being processed; there is no single, fixed reference point for the entire line. Instead, the delays are calculated relative to the time it would take for a wave to travel from the focal point (which is at the sample depth) directly to the i -th receiving element, effectively making the

beamformer focus on the fly.

Algorithmically, this is implemented by defining the focal radius vector $\mathbf{r}_{rf}$ to be a linearly increasing sequence that matches the radial distance of each time sample:

$$\mathbf{r}_{rf}[k] = R + \frac{k}{f_s} \cdot c. \quad (12)$$

Note that the factor of 2 is not present here because this vector is used to define the focal point for the *transmit* path in the delay calculation, which is a one-way trip in the context of the simulated emission. The round-trip is handled by the physical time-of-flight of the received signal.

The difference in image quality is profound. Fixed focusing creates a narrow region of sharp focus with degraded resolution elsewhere, while dynamic focusing produces a consistently sharp image at all depths, as the beamformer continuously adapts to the curvature of the incoming wavefronts. The trade-off is a significant increase in computational complexity, as unique delays must be calculated for every element and every sample, rather than for every element and just one focal depth per line.

3.4 Signal Interpolation for Fractional Delays

In digital beamforming, the calculated time delays τ_i correspond to continuous time, while the RF signals $s_i[n]$ are discrete. The required sample index, $d = \tau_i \cdot f_s$, is therefore non-integer. Applying the nearest-neighbor interpolation by simply rounding d introduces significant quantization error, degrading image quality through beamforming noise and reduced resolution.

To minimize this error, a linear interpolation scheme was implemented for its favorable trade-off between accuracy and computational complexity. For a desired fractional index d , the algorithm calculates the interpolated signal value $\hat{s}_i[d]$ using the two adjacent samples:

$$\hat{s}_i[d] = (1 - \alpha) \cdot s_i[\lfloor d \rfloor] + \alpha \cdot s_i[\lceil d \rceil] \quad (13)$$

where $\alpha = d - \lfloor d \rfloor$ is the fractional part. This process, illustrated in Figure 5, provides a more precise estimate of the analog signal value at the exact delay time than nearest-neighbor sampling. This interpolation is applied for every active element and every focal point during the beamforming summation, crucially improving the final image's sharpness and accuracy.

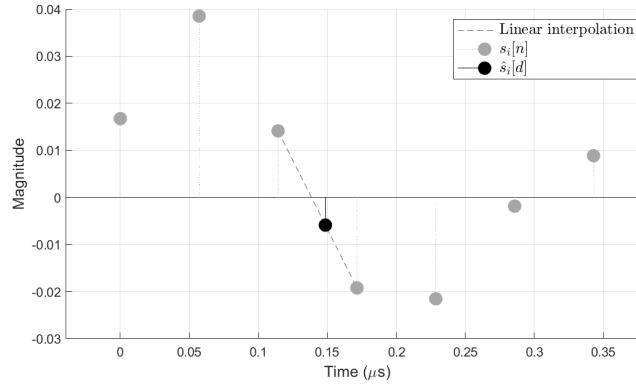


Figure 5: Visualization of linear interpolation for a fractional delay d . The value is estimated from the two nearest discrete samples.

3.5 Apodization

Apodization, in the context of ultrasound beamforming, refers to the application of a window function across the active receive aperture. This technique is employed to weight the contribution of each element's signal before summation. The primary purpose of apodization is to control the beam pattern of the array, specifically to reduce the amplitude of side lobes and grating lobes at the expense of a slight widening of the main lobe. Lower side lobes result in improved image contrast by reducing artifacts from off-axis scattering, while a wider main lobe slightly degrades lateral resolution.

In this work, the apodization is implemented by defining a set of weights w_i for the $N_{\text{active}} = 64$ elements in the receive aperture. The beamformed signal from Eq. (7) is modified as:

$$s_{\text{BF}}(t, \mathbf{r}_f) = \sum_{i=1}^{N_{\text{active}}} w_i \cdot s_i(t - \tau_i(\mathbf{r}_f)). \quad (14)$$

A family of symmetric window functions, known as generalized cosine windows, was implemented due to their effectiveness and computational simplicity. These windows are defined by the general formula:

$$w[n] = a_0 - a_1 \cos\left(\frac{2\pi n}{N-1}\right) + a_2 \cos\left(\frac{4\pi n}{N-1}\right), \quad \text{for } n = 0, 1, \dots, N-1, \quad (15)$$

where N is the window length and the coefficients a_0, a_1, a_2 determine the specific window type. The implemented windows and their corresponding coefficients are detailed in Table 2.

The choice of window involves a direct trade-off. The rectangular window (i.e., no apodization, all weights equal to 1) provides the narrowest main lobe and thus the best theoretical lateral resolution, but it also produces the highest side lobes. The Hann, Hamming, and Blackman windows progressively suppress the side lobes further, leading to better contrast, but at the cost of a broader main lobe and a consequent reduction in resolution.

In the algorithm, the function `win_apod` generates the specified window based on the

Table 2: Coefficients for Generalized Cosine Apodization Windows

Window Type	a_0	a_1	a_2
Rectangular	1.00	0.00	0.00
Hann	0.50	0.50	0.00
Hamming	0.54	0.46	0.00
Blackman	0.42	0.50	0.08

input string (e.g., ‘hann’) and the coefficients from Table 2. This window is then applied element-wise during the summation in the beamforming loop.

3.6 Envelope Detection and Log Compression

The beamformed signal $s_{BF}(t)$ is a radio-frequency (RF) signal. To form a standard B-mode image, its amplitude must be extracted and its large dynamic range compressed for display.

Envelope detection demodulates the RF signal to obtain the instantaneous amplitude. This is efficiently performed using the Hilbert transform to generate the analytic signal. The envelope for each scan line is calculated as:

$$\text{env}(t) = |\mathcal{H}\{s_{BF}(t)\}|, \quad (16)$$

implemented using MATLAB’s `hilbert` function.

The envelope’s dynamic range, which can exceed 100 dB, is compressed logarithmically to fit a grayscale display. The normalized envelope is converted to decibels (dB):

$$\text{img}_{dB} = 20 \cdot \log_{10} \left(\frac{\text{env}}{\max(\text{env})} \right). \quad (17)$$

A floor value of $10^{-60/20}$ is applied to clip signals below -60 dB, establishing a consistent 60 dB dynamic range for all images in this work.

3.7 Scan Conversion: Polar to Cartesian Interpolation

The beamformed and compressed data naturally resides in a polar coordinate system (r, θ) , where each scan line corresponds to a specific steering angle θ_{rf} and the samples along that line correspond to radial distances r . To display this data as a standard, rectilinear image, it must be transformed from this native polar grid to a Cartesian grid (x, z) . This process is known as scan conversion.

The relationship between the coordinate systems is given by the polar-to-Cartesian transformation:

$$\begin{aligned} x &= r \cdot \sin(\theta), \\ z &= r \cdot \cos(\theta). \end{aligned} \quad (18)$$

In the algorithm, the beamformed data is stored in a matrix where the columns represent the 129 scan lines (angles θ_{rf}) and the rows represent the samples (radial distances r). A regular Cartesian grid is defined for the desired image dimensions (1024 x 1024 pixels in this case). The coordinates of every pixel in this grid (X, Z) are converted to polar coordinates (Θ_o, R_o) using the inverse transformation.

The final image is generated by interpolating the beamformed data from the original (θ_{rf}, r) grid onto the new (Θ_o, R_o) grid. This was implemented using MATLAB's `interp2` function with linear interpolation. It is important to note that the axial coordinate z is adjusted in the final display to start at the transducer's surface rather than at the center of curvature, providing a more intuitive anatomical view.

3.8 Quantitative Analysis of Lateral Resolution

To quantitatively assess the performance of the beamformer as a function of depth, the lateral resolution was measured by calculating the Full Width at Half Maximum (FWHM) of the point spread function (PSF) for isolated point targets in the simulated phantom. The lateral resolution is defined as the width of the main lobe of the PSF at the -6 dB level relative to its peak, providing a standard metric for the system's ability to distinguish two closely spaced point scatterers in the lateral direction.

The algorithm, implemented in the function `fwhm_calc`, operates as follows. First, it identifies the axial positions of the point targets by finding local maxima along the central lateral line of the image to avoid off-axis degradation effects. For each detected wire at depth z_i , a lateral slice of the beamformed image $\mathbf{img}(z_i, x)$ is extracted.

Let $S(x) = \mathbf{img}(z_i, x)$ be this lateral slice. The algorithm finds the peak value $P_{\max}$ and its corresponding lateral position x_{peak} . The -6 dB threshold, T , is then calculated. Since the image is in decibels, the half-power point corresponds to -6 dB below the peak:

$$T = P_{\max} - 6 \text{ dB}. \quad (19)$$

The algorithm then locates the two lateral positions, x_{left} and x_{right} , where the profile $S(x)$ crosses this threshold. A linear interpolation scheme is employed between adjacent samples to achieve sub-pixel accuracy for these crossing points. For instance, to find x_{right} between sample indices j and $j + 1$, where $S(j) > T$ and $S(j + 1) \leq T$, the position is calculated as:

$$x_{\text{right}} = x[j] + \frac{T - S(j)}{S(j + 1) - S(j)} \cdot (x[j + 1] - x[j]). \quad (20)$$

The same procedure is applied to find x_{left} on the other side of the peak.

Finally, the FWHM is computed as the lateral distance between the two interpolated -6 dB crossing points:

$$\text{FWHM}_i = |x_{\text{right}} - x_{\text{left}}|. \quad (21)$$

This process, illustrated in Figure 6, is repeated for every detected point target in the phantom. The output is a set of depth-resolution pairs (**depths**, **fwhm**), which allows for the analysis of how parameters such as receive focusing and apodization affect the resolution

at different imaging depths. This provides an objective basis for comparing the performance of different beamforming configurations.

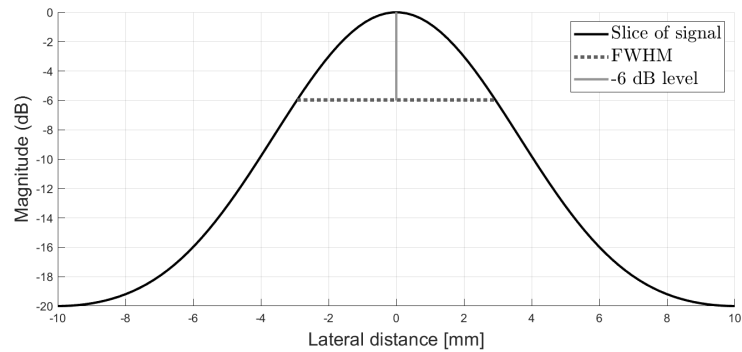


Figure 6: Schematic of the FWHM measurement on a lateral slice of a point target. The -6 dB level is determined relative to the peak magnitude. The FWHM is the distance between the left and right crossing points, found via linear interpolation for sub-pixel accuracy.

4 Results

This section presents the images and quantitative analysis resulting from the beamforming process detailed in the Methodology. The performance of the developed algorithm is first evaluated using the simulated wire phantom, providing a controlled environment to analyze the impact of key parameters like focusing method and apodization. Subsequently, the beamformer is applied to measured phantom data and an *in-vivo* liver capture to demonstrate its functionality on real-world datasets. All images are displayed with a consistent dynamic range of 60 dB.

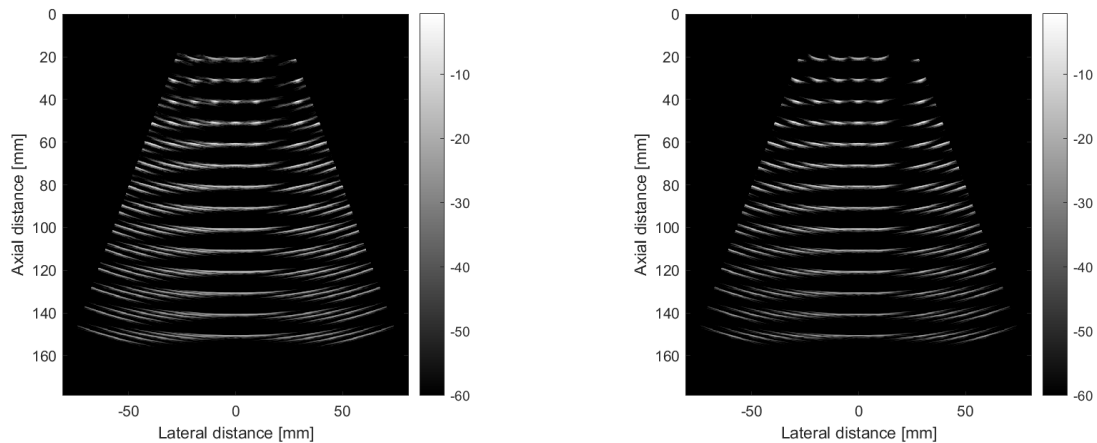
4.1 Beamforming of Simulated Point Phantom

The simulated wire phantom, consisting of a series of isolated point targets at various depths, serves as an ideal tool for quantifying beamformer performance. The following results illustrate the effects of receive focusing and apodization on image quality.

4.1.1 Impact of Receive Focusing Method

The fundamental difference between fixed and dynamic receive focusing is vividly demonstrated in Figure 7. (Figure 7a) shows the image formed using fixed receive focusing with a focal depth of 40 mm beyond the transducer surface and no apodization (rectangular window). A distinct region of sharp focus is visible around the 40-60 mm depth range, where the point targets appear as fine, well-defined dots. However, above and below this zone, the points become progressively blurred and elongated in the lateral direction, indicating a severe degradation of lateral resolution. This defocusing is the expected result of using a single, static set of time delays for the entire scan line.

In contrast, (Figure 7b), which was generated using dynamic receive focusing with the same rectangular apodization, shows a dramatic improvement. The point targets are consistently sharper and defined across the entire image depth, from the near field to the far field. The absence of defocusing artifacts confirms that the beamformer is successfully calculating and applying unique, depth-optimal delays for each sample, resulting in a better point spread function (PSF) throughout the image. This validates the theoretical advantage of dynamic focusing outlined in Section 3.3.2.



(a) Fixed focusing, Rectangular window. (b) Dynamic focusing, Rectangular window.

Figure 7: Comparison of fixed and dynamic receive focusing on the simulated point phantom. The fixed focus image is only sharp near its focal zone (40-60 mm), while the dynamic focus image maintains sharpness at all depths.

4.1.2 Impact of Apodization Window

Building upon dynamic focusing, the effect of different apodization windows on image quality is presented in Figure 8. All images in this figure were created with dynamic receive focusing, isolating the effect of the apodization function.

(Figure 8a) shows the result with a rectangular window. The point targets appear very narrow, suggesting excellent lateral resolution. However, this perceived sharpness is compromised by significant side lobes, visible as a pronounced ‘cross-hatch’ or high-level clutter pattern between the points. This clutter severely reduces image contrast and, as quantified in Section 4.1.3, can lead to an overestimation of the FWHM by interfering with the -6 dB threshold measurement.

(Figure 8b) demonstrates the effect of a Hann window. The most immediate improvement is the substantial suppression of side lobe clutter, resulting in a much cleaner background and superior contrast. Compared to the rectangular case, it offers a better resolution in the mid-to-far field by eliminating side lobe interference. This makes the Hann window a superior choice for a balanced image, offering both good contrast and reliable resolution.

(Figure 8c) shows a similar effect with the Hamming window, which offers a nearly identical trade-off to the Hann window, providing strong side lobe suppression and a clean point spread function.

Finally, (Figure 8d), using a Blackman window, exhibits the cleanest background of all, with side lobes almost entirely eliminated. However, this aggressive apodization comes with the most significant main lobe broadening. This broadening is visibly apparent in the image, where the point targets appear the widest and most diffuse. The quantitative data confirms this, showing that the Blackman window’s FWHM is generally worse than the Hann and Hamming windows, and it can fail entirely in the far field where the main lobe becomes

excessively broad. This visual and quantitative evidence perfectly illustrates the critical trade-off between main lobe width (resolution) and side lobe level (contrast).

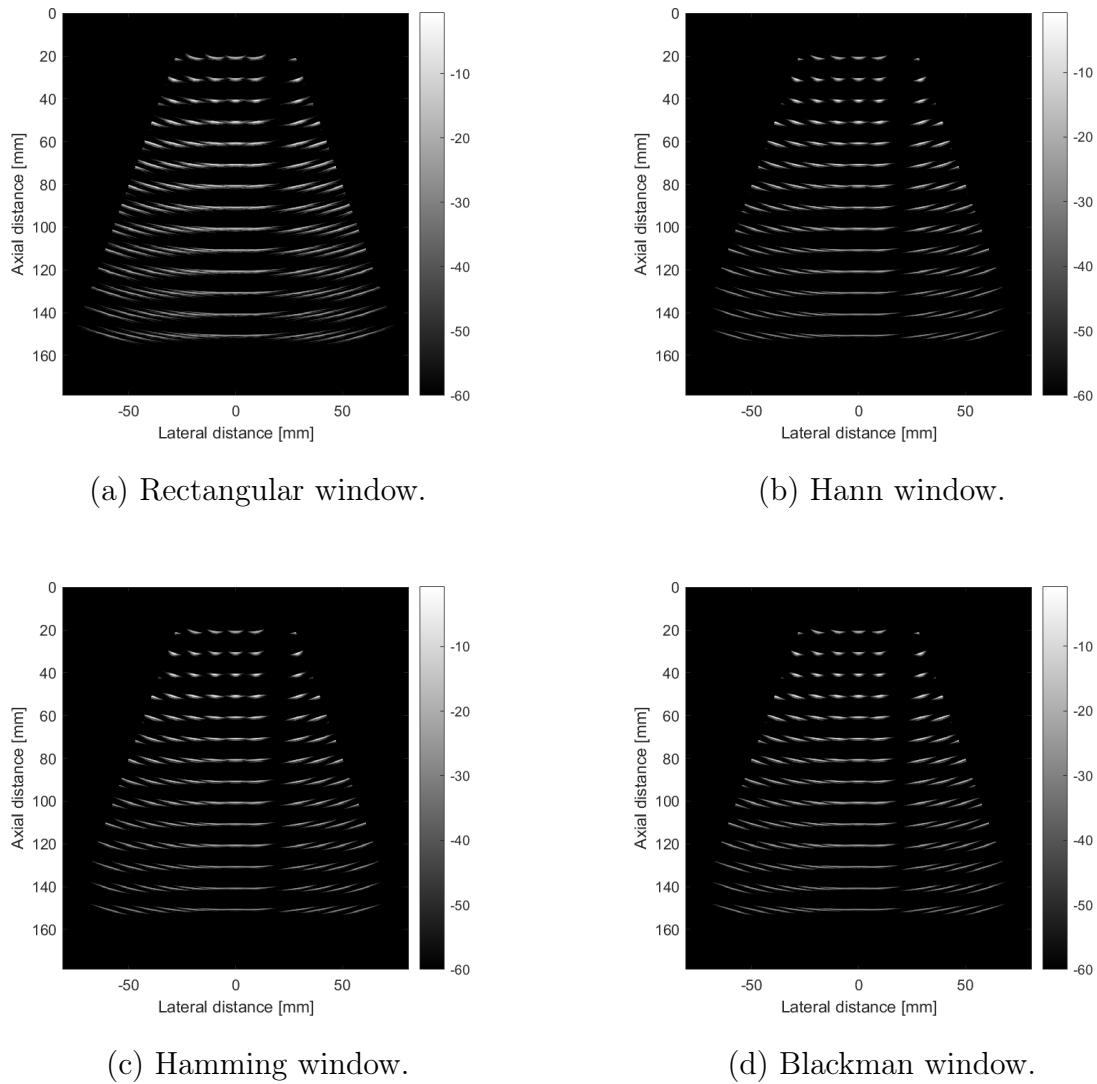


Figure 8: Impact of apodization windows on the simulated point phantom with dynamic focusing. The rectangular window (a) offers the sharpest points but high side lobe clutter. The Hann (b), Hamming (c), and Blackman (d) windows progressively suppress side lobes at the cost of broader main lobes.

4.1.3 Quantitative Lateral Resolution Analysis

The qualitative observations from the images are quantified by measuring the lateral resolution as a function of depth using the Full Width at Half Maximum (FWHM) of the point spread functions. The results for several key beamforming configurations are plotted in Figure 9.

The plot unequivocally demonstrates the superior performance of dynamic focusing. The **Dynamic-Rectangular** configuration provides the best possible resolution across most depths, maintaining an FWHM below 8 mm until a depth of approximately 110 mm. In stark contrast, the **Fixed-Rectangular** configuration shows a catastrophic degradation of resolution away from its focal zone. While it achieves an excellent FWHM of 0.99 mm at 30 mm depth, its performance deteriorates rapidly, exceeding 20 mm in the far field. This quantifies the severe defocusing artifact observed in Figure 7a, rendering the image diagnostically useless outside a narrow focal region.

The effect of apodization presents a more nuanced trade-off. The FWHM measurement can be corrupted by high side lobes, which raise the signal level around the -6 dB threshold and cause an overestimation of the main lobe's width. By suppressing these side lobes, apodization provides a cleaner and more accurate measurement of the true main lobe.

This effect is clearly visible in the data. For example, at a depth of 120 mm, the Dynamic-Rectangular FWHM is 8.29 mm, while the Dynamic-Hann and Dynamic-Hamming FWHMs are 6.40 mm and 6.51 mm, respectively. Here, the apodized windows report a better (lower) FWHM because they have successfully mitigated the side lobe interference that plagues the rectangular window.

However, this benefit reaches a limit. The **Dynamic-Blackman** window, which applies the most aggressive side lobe suppression, demonstrates this trade-off. In the mid-field (e.g., 80 mm), it performs similarly to the Hann and Hamming windows. However, in the far field, where the main lobe has naturally broadened due to diffraction, the aggressive apodization of the Blackman window exacerbates this broadening. The main lobe becomes so wide that the signal from a point target falls below the -6 dB threshold relative to the peak, causing the FWHM algorithm to fail or report an anomalously high value, as seen at 150 mm with an FWHM of 24.7 mm. At this point, the Blackman window has degraded the resolution to the point where it can no longer reliably distinguish between closely spaced objects, despite having the cleanest background.

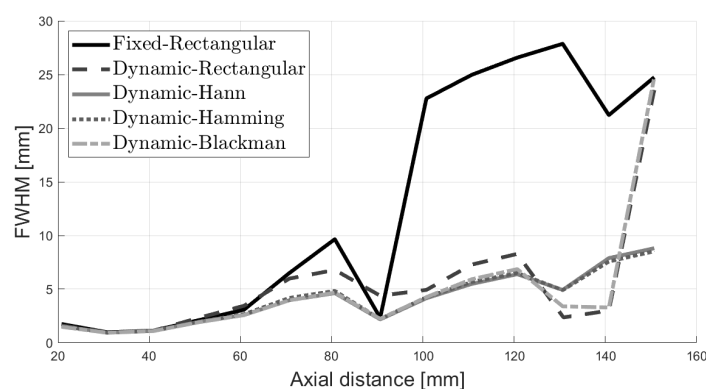


Figure 9: Lateral resolution (FWHM) as a function of depth for different beamforming configurations. Dynamic focusing provides consistent performance, while fixed focusing fails outside its focal zone. Apodization (Hann, Hamming) improves measured resolution in the mid-to-far field by suppressing side lobes, but aggressive apodization (Blackman) can cause failure in the far field due to excessive main lobe broadening.

4.2 Beamforming of Measured Phantom Data

To validate the beamformer on physical data, the algorithm was applied to measured signals from a tissue-mimicking phantom. The resulting image, generated using dynamic receive focusing and a Hann apodization window, is presented in Figure 10.

The beamformer successfully reconstructs the underlying structure of the phantom. Several hyperechoic point targets are clearly visible and well-defined, demonstrating the system's ability to resolve structures with different scattering properties. The overall image quality is good, with a characteristic speckle pattern in the tissue-mimicking regions.

However, a notable difference from the simulated data is the presence of reverberation artifacts, visible as repeating horizontal bands at constant depths below strong reflectors. These artifacts are not a failure of the beamforming algorithm but are inherent to the physical measurement process. They occur due to multiple reflections between highly reflective surfaces in the experimental setup, such as between the transducer face and a strong scatterer, which are not modeled in the ideal simulated environment. The presence of these artifacts confirms that the beamformer is accurately processing real acoustic wave propagation phenomena, including these unwanted multipath signals.

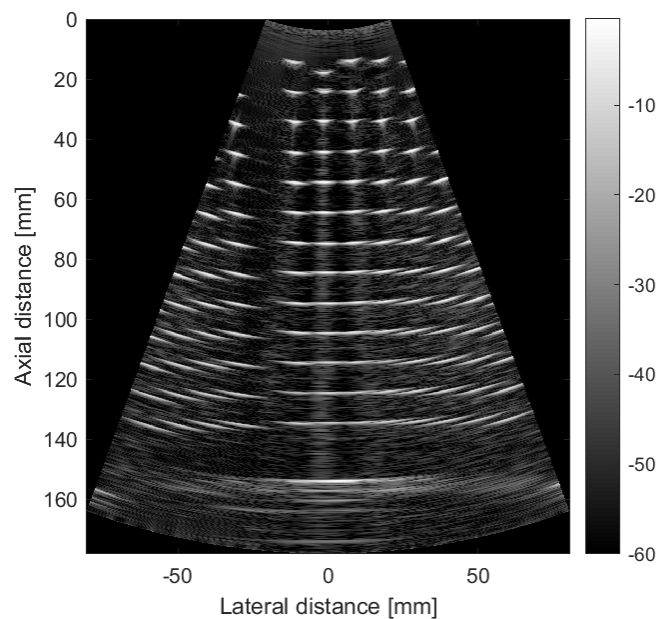


Figure 10: Beamformed image of the measured phantom data using dynamic focusing and a Hann window. The image shows hyperechoic targets, and characteristic speckle. Reverberation artifacts appear as horizontal bands below strong reflectors.

4.3 Beamforming of *In-Vivo* Liver Data

Finally, the clinical relevance of the beamformer is demonstrated by processing an *in-vivo* capture of a human liver. A single frame, beamformed with dynamic focusing and a

Hann window, is shown in Figure 11.

The image presents a classic subcostal view of the liver. The liver parenchyma itself displays a homogeneous, fine speckle pattern, which is the expected appearance for normal liver tissue. The most prominent feature is the portal vein, which appears as a large, anechoic (black) tubular structure due to the blood it contains, which is a weak scatterer of ultrasound. The bright, hyperechoic walls of the portal vein are delineated against the darker lumen and the surrounding parenchyma.

Other finer structures, such as smaller branching vessels and the liver capsule, can also be discerned. The overall image exhibits good contrast, allowing for clear differentiation between the fluid-filled vessels and the scattering tissue. The successful visualization of these anatomical structures confirms that the implemented delay-and-sum beamformer, with appropriate focusing and apodization, is capable of producing diagnostically useful images from clinical ultrasound data.

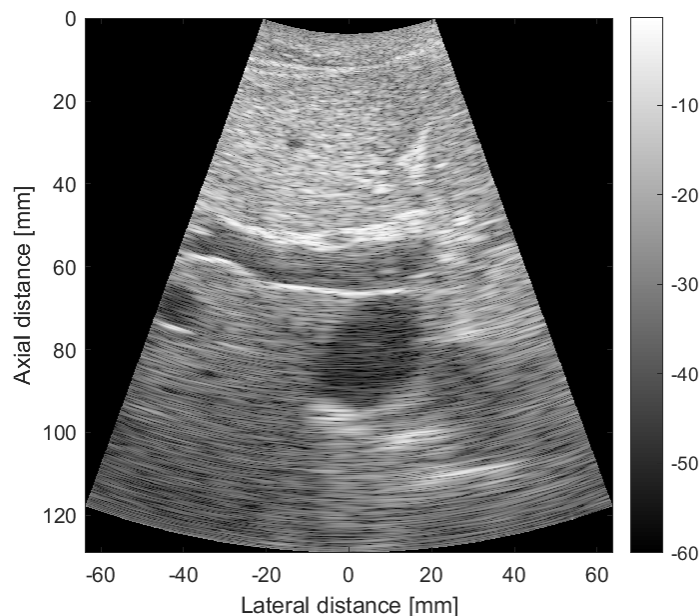


Figure 11: Beamformed image of a single frame of *in-vivo* liver data using dynamic focusing and a Hann window. The image shows the liver parenchyma with its characteristic speckle, the anechoic portal vein (PV) with bright walls, and other vascular structures.

5 Discussion

The results presented in the previous section comprehensively demonstrate the functionality and performance of the implemented convex array beamformer. The analysis validates theoretical expectations and provides practical insights into the trade-offs inherent in ultrasound image formation.

Dynamic receive focusing proved essential for diagnostic-quality imaging. Fixed focusing produced severe resolution degradation outside its focal zone (Figure 7a), as quantified by exponentially growing FWHM values in Figure 9, making it clinically inadequate.

Apodization analysis revealed nuanced trade-offs. While rectangular windows provided theoretical maximum resolution, their high side lobes corrupted both image contrast and FWHM accuracy. Hann and Hamming windows emerged as optimal, balancing excellent side lobe suppression with reliable resolution. The Blackman window, though providing superior contrast, suffered from excessive main lobe broadening that compromised far-field resolution.

A key observation from the FWHM analysis is the inherent degradation of lateral resolution with depth.. This is a fundamental physical limitation governed by the effective F-number ($F\#$), defined as the ratio of depth z to the aperture size D . In this work, the receive aperture was fixed at 64 active elements, meaning the physical aperture size D was constant. As depth increases, the $F\# = z/D$ consequently increases, leading to a broader diffraction-limited main lobe and thus a larger FWHM. The steady rise in the FWHM for the Dynamic-Rectangular configuration in Figure 9 is a direct manifestation of this increasing F-number. To maintain a constant F-number and hence a constant lateral resolution with depth, the aperture should be dynamically expanded with depth, a technique not implemented in this beamformer but which represents a logical next step for performance enhancement.

The beamformer demonstrated robust performance across data types. The presence of reverberation artifacts, while undesirable, correctly indicates that the beamformer is processing genuine acoustic phenomena. Finally, the *in-vivo* liver image stands as the ultimate validation of the project. The clear delineation of anatomical structures like the portal veins and the characteristic liver parenchyma demonstrates that the implemented DAS beamformer is fully capable of producing images of diagnostic quality.

In summary, the combination of dynamic focusing with a balanced apodization window, such as Hann or Hamming, was identified as the key configuration for achieving high-quality ultrasound imaging with a convex array. This configuration ensures uniform lateral resolution, high contrast, and the clean visualization of anatomical detail necessary for clinical assessment. The results also highlight the fundamental role of the F-number in determining resolution, suggesting that dynamic aperture control could be a key area for future improvement.

6 Conclusion

This project centered on the implementation and analysis of a Delay-and-Sum beamformer for convex array ultrasound. We successfully developed a functional beamformer in MATLAB and systematically evaluated its performance.

We achieved a clear understanding of the critical parameters governing image quality. The necessity of dynamic focusing for uniform resolution was unequivocally proven, and the nuanced role of apodization was quantified, identifying the Hann window as an optimal compromise between contrast and resolution.

The key gain from this work is a validated, end-to-end beamforming pipeline. The algorithm was proven robust, producing high-quality images from simulated, measured, and *in-vivo* data. The quantitative FWHM analysis provided concrete evidence of the system's performance and its physical limitations, offering a practical foundation for future enhancements in ultrasound image formation.

7 Appendix

Listing 1: MATLAB Script implemented, which includes the main code and the custom functions used

```
1  clc
2  clear all
3  close all
4  %% Execution
5  % Beamforming and plotting
6  [img1, x1, z1] = beamform(1, 'rect', false, 60); % Fixed Focus
7
8  [img2, x2, z2] = beamform(1, 'rect', true, 60); % Dynamic Focus —
    Rectangular
9
10 [img3, x3, z3] = beamform(1, 'hann', true, 60); % Dynamic Focus — Hann
11
12 [img4, x4, z4] = beamform(1, 'hamming', true, 60); % Dynamic Focus — Hamming
13
14 [img5, x5, z5] = beamform(1, 'blackmann', true, 60); % Dynamic Focus —
    Blackmann
15
16 % Calculation of FWHM
17 [depth1, fwh1] = fwhm_calc(img1, x1, z1); % Fixed Focus
18 [depth2, fwh2] = fwhm_calc(img2, x2, z2); % Dynamic Focus — Rectangular
19 [depth3, fwh3] = fwhm_calc(img3, x3, z3); % Dynamic Focus — Hann
20 [depth4, fwh4] = fwhm_calc(img4, x4, z4); % Dynamic Focus — Hamming
21 [depth5, fwh5] = fwhm_calc(img5, x5, z5); % Dynamic Focus — Blackmann
22
23 % Plotting FWHM
24 figure;
25 hold on
26 plot(depth1,fwh1,"Color","k","LineWidth",3)
27 plot(depth2,fwh2,"Color",[0.25, 0.25, 0.25],"LineStyle","—","LineWidth",3)
28 plot(depth3,fwh3,"Color",[0.50, 0.50, 0.50],"LineWidth",3)
29 plot(depth4,fwh4,"Color",[0.37, 0.37, 0.37],"LineStyle",":","LineWidth",3)
30 plot(depth5,fwh5,"Color",[0.65, 0.65, 0.65],"LineStyle","-.", "LineWidth",3)
31 x = xlabel('Axial distance [mm]');
32 y = ylabel('FWHM [mm]');
33 lgd = legend("Fixed-Rectangular","Dynamic-Rectangular","Dynamic-Hann","
    Dynamic-Hamming","Dynamic-Blackman","Location","northwest");
34 lgd.FontSize = 16;
35 x.FontSize = 16;
36 y.FontSize = 16;
```

```
37 grid
38
39 % Beamforming of other data sets
40 beamform(2, 'hann', true, 60); % Phantom measured
41
42 beamform(3, 'hann', true, 60); % In-vivo
43
44 %% Functions
45 function [img, x, z] = beamform(id_data, id_apod, id_focus, db_level)
46 %beamform    Create the image resulted by beamforming ultrasound data of a
    convex array
47 %
48 %USAGE
49 %    [img, x, z] = beamform(id_data)
50 %    [img, x, z] = beamform(id_data, id_apod, id_focus, db_level)
51 %
52 %INPUT ARGUMENTS
53 % id_data    : integer specifying selection of data to beamform
54 %             1 = simulated data
55 %             2 = phantom data
56 %             3 = in-vivo data
57 % id_apod    : string specifying window type for apodization (default, type =
    'hamming')
58 %             'rect'      = rectangular window (no apodization)
59 %             'hann'      = hann window
60 %             'hamming'   = hamming window
61 %             'blackmann' = blackmann window
62 % id_focus   : boolean specifying approach to focusing (default, id_focus =
    true)
63 %             true  = dynamic receive focusing
64 %             false = fixed receive focusing
65 % db_level   : integer specifying db level for compression (default, db_level
    = 60)
66 %
67 %OUTPUT ARGUMENTS
68 %    img : resulting image [1024 x 1024]
69 %    x   : vector containing lateral coordinates [1024 x 1]
70 %    z   : vector containing axial coordinates [1024 x 1]
71 %
72 %    beamform(...) plots the image in a new figure.
73 %% CHECK INPUT ARGUMENTS
74 % Check for proper input arguments
75 if nargin < 1 || nargin > 4
76     error('Wrong number of input arguments!')
```

```
77 end
78
79 % Set default values
80 if nargin < 4 || isempty(db_level); db_level = 60; end
81 if nargin < 3 || isempty(id_focus); id_focus = true; end
82 if nargin < 2 || isempty(id_apod); id_apod = 'hamming'; end
83
84 %% Initializing requirements for data
85
86 switch id_data % Determining folder path for obtaining data
87     case 1 % Simulated data for convex array
88         file_path = "phantom_matrix_sim_files\duplex\B_mode\seq_0001\";
89     case 2 % Measured data for convex array and matrix phantom
90         file_path = "phantom_matrix_files\duplex\B_mode\seq_0001\";
91     case 3 % Measured in-vivo data for convex array
92         file_path = "convex_array_files\B_mode\seq_0001\";
93     otherwise
94         error('Id "%d" is not supported',id_data);
95 end
96
97 % Loading first file to initialize some general parameters with its size
98 tag = file_path + "elem_data_em0" + num2str(1,"%.3d") + ".mat"; % General
    format for files
99 load(tag)
100
101 %% Setting general parameters
102
103 n = size(samples,1); % Number of data points recorded by each transducer per
    emission
104
105 m = 129; % Total amount of emissions
106 c = 1491; % Speed of sound (m/s)
107 fs = 17.5E6; % Sampling frequency (Hz)
108
109 n_elements = 192; % Number of elements (transducers)
110 width = 0.30E-3; % Width of one element (m)
111 kerf = 0.03E-3; % Kerf (gap) between elements (m)
112 radius = 60.3E-3; % Convex radius (m)
113
114 %% Defining transducer geometry
115 % Calculating the element pitch
116 pitch = width + kerf;
117 % Obtain the total length of the convex element arc
118 total_arc_length = n_elements * pitch;
```

```
119 % Calculate the total angle which corresponds to the ratio of the length of
120 % the element arc and the convex radius
121 total_angle = total_arc_length / radius;
122
123 % Initializing variables to calculate the relative position of each element
124 theta_i = zeros(n_elements,1); % Variable that stores the angles of each
    element
125 r_i = zeros(n_elements,2); % Variable that stores the positions (x, z) of
    each element
126
127 % Cycle to calculate positions of each element
128 for j = 1:1:n_elements
129     theta_i(j) = -total_angle/2 + (j-0.5) * (total_angle/192); % Calculate
        the angle corresponding to the i'th element
130     r_i(j,1) = radius * sin(theta_i(j)); % Obtain x coordinate for the i'th
        element
131     r_i(j,2) = radius * cos(theta_i(j)); % Obtain z coordinate for the i'th
        element
132 end
133
134 %% Focus preparation
135 % In this section it is decided if the approach is going to be the one
136 % which has delay oriented calculation or the one which finds each point
137 % individually
138
139 if id_focus % Dynamic receive focusing
140     r_rf = (1:1:n) / fs * c + radius; % Creation of vector for different
        radius of focal points
141 else % Fixed receive focusing
142     r_rf = zeros(1,n) + radius + 40E3; % Creation of vector for fixed focal
        point (40 mm)
143 end
144
145 %% Apodization preparation
146 % Obtaining the apodization window for the fixed active probe elements
147 n_w = 64; % Active probe elements
148 w = win_apod(n_w, id_apod); % Calculation of window
149
150 %% Beamforming
151
152 % Initializing variables
153 s_t = zeros(n,m); % Matrix with beamformed data
154 r_f = [0,0]; % Vector with position of focal point
155 r_c = [0,0]; % Vector with position of beam reference point
```

```
156
157 for j = 1:1:m % Iterating through number of emissions
158     % Loading j'th file
159     tag = file_path + "elem_data_em0" + num2str(j, "%.3d") + ".mat"; %
        General format for files
160     load(tag)
161     % Standarizing format, particularly for cases when data is in uint16
162     samples = double(samples);
163     % Center data around zero, baseline removal
164     baseline = mean(samples);
165     samples = samples - baseline;
166
167     % Obtaining position of beam reference point according to the emission
168     theta_rf = mean(theta_i(j:j+63)); % Obtain mean angle of the 64 emitting
        elements
169     r_c(1) = radius * sin(theta_rf); % Obtain x coordinate for the j'th beam
        reference
170     r_c(2) = radius * cos(theta_rf); % Obtain z coordinate for the j'th beam
        reference
171
172     for k = 1:1:n % Iterating through all the focal points previously
        defined
173         % Obtaining position of focal point
174         r_f(1) = r_rf(k) * sin(theta_rf); % Obtain x coordinate for the k'th
            focal point
175         r_f(2) = r_rf(k) * cos(theta_rf); % Obtain z coordinate for the k'th
            focal point
176
177         for i = j:j+n_w-1 % Iterating through active elements, window of 64
            elements per emission
178
179             % Calculating time delay for current element
180             d_fi = norm(r_f - r_i(i,:)); % Calculating the distance from the
                focal point to the i'th element
181             % The next is only done when fixed focusing, due to making the
182             % delay relative to another element
183             d_fc = norm(r_f - r_c) * ~id_focus; % Calculating the distance
                from the focal point to the beam reference
184             tau_i = (d_fi - d_fc) / c; % Converting distance to time
185             sample_delay = tau_i * fs + k * ~id_focus; % Converting time to
                sample index
186             % The term "k" is added in each iteration to effectively cycle
187             % through the different samples when fixed receiving focal
188             % point is selected
```

```
189
190     % Due to the limitation in resolution by to the sampling
191     % frequency a linear interpolation approach was made to
192     % compensate and obtain a better precision
193
194     % Start of the interpolation
195     idx_floor = floor(sample_delay); % Truncate to lower sample
        index
196     idx_ceil = ceil(sample_delay); % Truncate to upper sample index
197     frac = sample_delay - idx_floor; % Obtain decimal value of the
        sample index
198
199     if idx_ceil <= size(samples,1) % Sanity check to always obtain
        real values
200         if idx_floor == idx_ceil % In this case there is no fraction
            , so it landed on an integer index
201             temp = samples(idx_floor, i); % Obtain value of the
                signal corresponding to that sample
202         else % If there exists a fraction, it means the actual point
            is located in between samples
203             % The equation of a straight line is followed to
204             % make an approximation for the actual value
205             % corresponding to that particular time
206             temp = (1-frac)*samples(idx_floor,i) + frac*samples(
                idx_ceil,i); % Obtain value of the signal
                corresponding to that sample
207         end
208         s_t(k, j) = s_t(k, j) + temp * w(i-j+1); % Multiplying data
            by apodization window and adding the line of the signal
            to the whole sum
209     end
210 end
211 end
212 end
213
214 %% Envelope and compression of signal
215 env_data = zeros(size(s_t)); % Initializing variable for envelope
        calculation
216
217 for j = 1:1:m % Calculating for each line of signal
218     env_data(:,j) = abs(hilbert(s_t(:,j))); % Obtaining the envelope through
        hilbert transform
219 end
220
```

```

221 % Realizing the compression of the signal
222 norm_RFdata = env_data / max(env_data(:)); % Normalizing it by its max value
223 floor_linear = 10^(-db_level / 20); % Setting the floor for compression
224 compressed_signal = max(norm_RFdata, floor_linear); % Compressing the signal
225 % Converting it to logarithmic scale
226 compressed_dB = 20 * log10(compressed_signal);
227
228 %% Interpolation
229 % Obtaining the distance vector in radial shape according to the speed of
230 % sound, the sampling frequency and the amount of samples
231 r = radius + (1:size(compressed_dB,1)) / fs * c / 2;
232
233 % Obtaining the vector with the angles of each emission beam, which
234 % corresponds to the angles of the calculated signal lines
235 for j = 1:1:129 % Calculating for each line of signal
236     theta_rf(j) = mean(theta_i(j:j+63)); % Obtain mean angle of the 64
        emitting elements
237 end
238
239 [Theta, R] = meshgrid(theta_rf, r); % Obtaining the grid for the
        interpolation in polar coordinates
240
241 % Defining the cartesian coordinates
242 x_l = r(end) * sin(theta_rf(1)); % Max lateral distance
243 z_i = radius * cos(theta_rf(1)); % Min axial distance (Since first element
        of transducer)
244 z_o = r(end); % Max axial distance
245
246 x = linspace(-x_l, x_l, 1024); % Vector containing lateral coordinates
247 z = linspace(z_i, z_o, 1024); % Vector containing axial coordinates
248
249 [X, Z] = meshgrid(x,z); % Obtaining the grid for the interpolation in
        cartesian coordinates
250
251 [Tho, Ro] = cart2pol(X,Z); % Transforming cartesian coordinates to polar
252
253 img = interp2(Theta, R, compressed_dB, -Tho + pi/2, Ro); % Performing
        interpolation
254
255 img(isnan(img)) = -60; % Setting pixels without value to the floor level
256
257 x = x * 1000; % Adjusting lateral vector to milimeter scale
258 z = (z - z_i) * 1000; % Adjusting axial vector to milimeter scale and
        adjusting the values so it starts at the height of the first element

```

```
259
260 %% Plotting Image
261
262 figure; % Plotting the image
263 imagesc(x, z, img);
264 axis equal; % Adjusting the scale
265 axis image;
266 xlabel('Lateral distance [mm]');
267 ylabel('Axial distance [mm]');
268 colormap(gray(128))
269 colorbar; % Displaying the colorbar
270 end
271
272 function w = win_apod(N, type)
273 %win_apod    Create apodization window
274 %
275 %USAGE
276 %      w = win_apod(N, type)
277 %
278 %INPUT ARGUMENTS
279 %      N : integer specifying selection size of window
280 % type : string specifying window type for apodization
281 %      'rect'      = rectangular window (no apodization)
282 %      'hann'      = hann window
283 %      'hamming'   = hamming window
284 %      'blackmann' = blackmann window
285 %
286 %OUTPUT ARGUMENTS
287 %      w : resulting apodization window [N x 1]
288 %
289 %% Modify parameter according to window type
290 switch lower(type)
291     case 'rect' % No apodization applied
292         a = 1;
293         b = 0;
294         c = 0;
295     case 'hann'
296         a = 0.5;
297         b = 0.5;
298         c = 0;
299     case 'hamming'
300         a = 0.54;
301         b = 0.46;
302         c = 0;
```

```
303     case 'blackmann'
304         a = 0.42;
305         b = 0.5;
306         c = 0.08;
307     otherwise
308         error('Window type "%s" is not supported',lower(type));
309 end
310
311 % Sample indices
312 n = (0:N-1)';
313 % Generalized form of cosine windows
314 w = a - b * cos(2*pi*n/(N-1)) + c * cos(4*pi*n/(N-1));
315 end
316
317 function [depths, fwhm] = fwhm_calc(img, x, z)
318 %fwhm_calc    Calculate Full Width at Half Maximum of wire targets
319 %
320 %USAGE
321 %    [depths, fwhm] = fwhm_calc(img, x, z)
322 %
323 %INPUT ARGUMENTS
324 %    img : beamformed image [N x N]
325 %    x : lateral coordinate vector [1 x N]
326 %    z : axial coordinate vector [1 x N]
327 %
328 %OUTPUT ARGUMENTS
329 %    depths : axial positions of detected wires [M x 1]
330 %    fwhm : lateral resolution measurements at each depth [M x 1]
331 %
332 %% INITIALIZE PARAMETERS
333 N = size(img,1); % Get number of axial samples
334
335 % Calculate -6 dB threshold for half maximum in logarithmic scale
336 half_dB = 20 * log10(0.5);
337
338 %% DETECT WIRE PEAKS
339 % Find local maxima along center lateral line to identify wire positions
340 [~, idx] = findpeaks(img(:,N/2),'MinPeakDistance',20);
341
342 M = length(idx); % Number of detected wires
343
344 % Initialize output arrays
345 fwhm = zeros(M,1);
346 depths = zeros(M,1);
```

```
347
348 %% MEASURE FWHM FOR EACH WIRE
349 for i = 1:M
350     % Extract lateral slice at current wire depth
351     slice = img(idx(i),:);
352
353     % Find peak magnitude and position in lateral direction
354     [mag_center, idx_center] = max(slice);
355
356     % Calculate -6 dB threshold relative to peak
357     threshold = mag_center + half_dB;
358
359     %% FIND THRESHOLD CROSSINGS
360     % Scan right from peak to find where signal drops below threshold
361     idx_right = idx_center;
362     while idx_right < length(slice) && slice(idx_right) > threshold
363         idx_right = idx_right + 1;
364     end
365     % Scan left from peak to find where signal drops below threshold
366     idx_left = idx_center;
367     while idx_left > 1 && slice(idx_left) > threshold
368         idx_left = idx_left - 1;
369     end
370
371     %% INTERPOLATE CROSSING POSITIONS
372     % Linear interpolation for right crossing between adjacent samples
373     t = (threshold - slice(idx_right-1)) / (slice(idx_right) - slice(
374         idx_right-1));
375     x_right = x(idx_right-1) + t * (x(idx_right) - x(idx_right-1));
376
377     % Linear interpolation for left crossing between adjacent samples
378     t = (threshold - slice(idx_left+1)) / (slice(idx_left) - slice(idx_left
379         +1));
380     x_left = x(idx_left+1) + t * (x(idx_left) - x(idx_left+1));
381
382     %% CALCULATE FWHM
383     % Compute lateral distance between -6 dB points
384     fwhm(i) = abs(x_right - x_left);
385
386     % Store corresponding depth of current wire
387     depths(i) = z(idx(i));
388 end
389 end
```